

VxLEARN Networks

Networking & Cybersecurity Track
Simulated Employment Program

Lab Report: The Client Interaction

Prepared by:
Kudzaishe Majeza
Junior Network Engineer – VxLEARN Networks

Mentor:
Titus Majeza
Senior Network Engineer

Date: 17 November 2025

Table of Contents

1. Introduction
2. Network Topology (Before Simulation)
3. Part 1 – Entering Simulation Mode
4. Part 2 – Setting Event List Filters
5. Part 3 – Requesting a Web Page
6. Part 4 – Running the Simulation
7. Part 5 – Inspecting the PDU
8. Part 6 – Understanding DNS + HTTP Interaction
9. Summary & Reflection

1. Introduction

This lab demonstrates how a client PC interacts with a server when requesting a web page.

Packet Tracer simulates the complete exchange:

- DNS lookup (resolving www.example.com to an IP)
- HTTP request sent from PC to server
- HTTP response sent back to PC
- TCP acknowledgements
- Segmented web data transfer

Understanding these interactions is fundamental in networking because nearly all modern communication relies on these two services: DNS to find the server, and HTTP to retrieve content.

2. Network Topology (Before Simulation)

The network is extremely simple:

- One PC
- One Server
- Directly connected with a copper cable
- The server runs DNS and HTTP services
- The PC uses the server for DNS resolution

This model helps isolate and observe exactly how URL resolution and webpage loading occur.



Figure 1: The simple network topology for client–server interaction demonstration.

3. Part 1 – Enter Simulation Mode

Simulation Mode allows Packet Tracer to show the **exact packets, protocols, and Layer-by-Layer OSI processing**.

Steps:

1. Click **Simulation Mode** in the bottom-right corner.
2. The event-list panel becomes visible.
3. Packet events will now pause and wait for manual forwarding.



Figure 2: Simulation mode activated, showing the Event List panel.

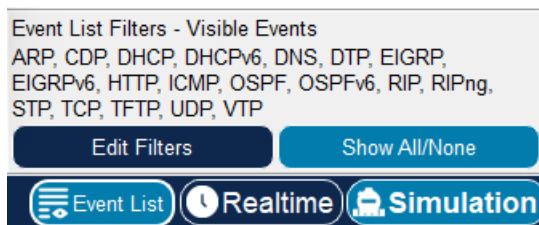
4. Part 2 - Set Event List Filters

The default setting shows **all** packet types, which overwhelms the screen.
We only want:

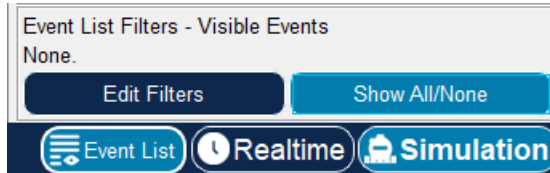
- **DNS packets**
- **HTTP packets**

Steps:

1. Click **Show All/None** → **None**, clearing all protocols.

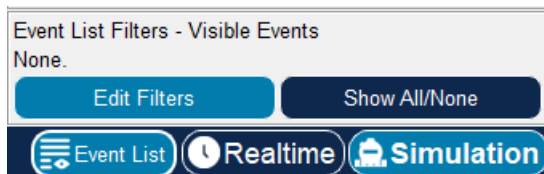


Before

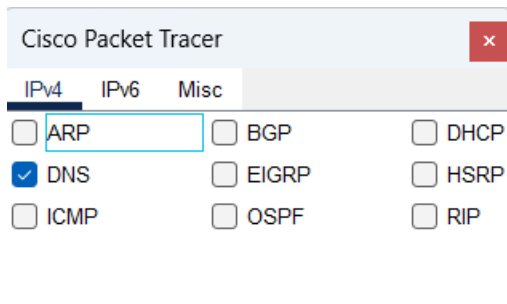


After

2. Click **Edit Filters**.



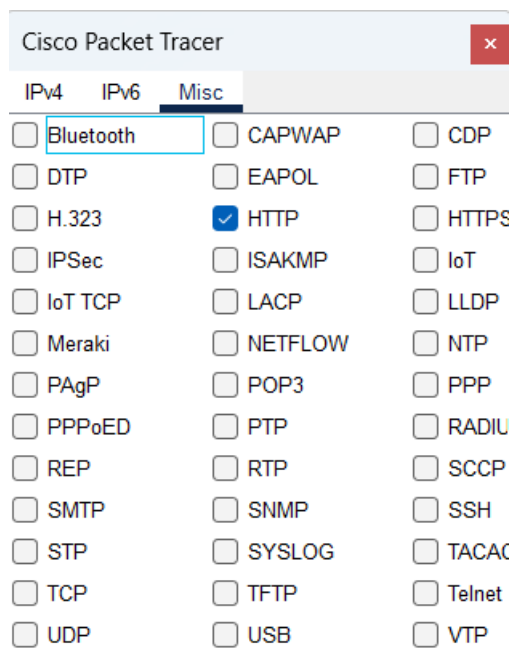
3. Under **IPv4**, select **DNS**.



The screenshot shows the 'Cisco Packet Tracer' window with the 'IPv4' tab selected. The 'DNS' checkbox is checked, and the 'ARP' checkbox is highlighted with a blue border. Other checkboxes include BGP, DHCP, EIGRP, HSRP, ICMP, OSPF, and RIP.

IPv4	IPv6	Misc
☐ ARP		☐ BGP
☒ DNS		☐ DHCP
☐ ICMP		☐ EIGRP
		☐ HSRP
		☐ OSPF
		☐ RIP

4. Under **Misc**, select **HTTP**.



The screenshot shows the 'Cisco Packet Tracer' window with the 'Misc' tab selected. The 'HTTP' checkbox is checked, and the 'Bluetooth' checkbox is highlighted with a blue border. Other checkboxes include CAPWAP, CDP, DTP, EAPOL, FTP, H.323, HTTPS, IPSec, IoT, LACP, LLDP, Meraki, NETFLOW, NTP, PAgP, POP3, PPP, PPPoED, PTP, RADIUS, REP, RTP, SCCP, SMTP, SNMP, SSH, STP, SYSLOG, TACACS, TCP, TFTP, Telnet, UDP, USB, and VTP.

IPv4	IPv6	Misc
☐ Bluetooth		☐ CAPWAP
☐ DTP		☐ CDP
☐ H.323		☒ HTTP
☐ IPSec		☐ HTTPS
☐ IoT TCP		☐ IoT
☐ Meraki		☐ LACP
☐ PAgP		☐ LLDP
☐ PPPoED		☐ NETFLOW
☐ REP		☐ NTP
☐ SMTP		☐ POP3
☐ STP		☐ PPP
☐ TCP		☐ PTP
☐ UDP		☐ RADIUS
		☐ REP
		☐ RTP
		☐ SCCP
		☐ SNMP
		☐ SSH
		☐ SYSLOG
		☐ TACACS
		☐ TFTP
		☐ Telnet
		☐ USB
		☐ VTP

5. Close.

Now you will only see the important packets related to the client's web activity.

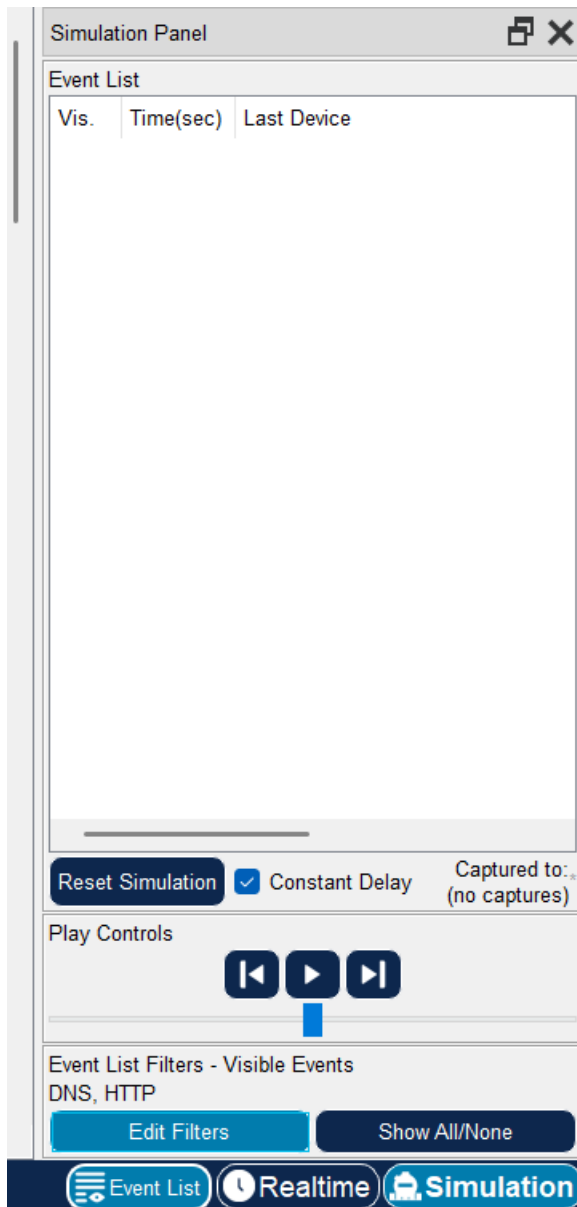


Figure 3: Event List Filters configured to display only DNS and HTTP packets.

5. Part 3 – Requesting a Web Page

Steps:

1. Click **PC** → **Desktop** → **Web Browser**.
2. Enter:

`www.example.com`

3. Click **Go**.
4. Minimize (do not close) the browser.

This triggers:

- A DNS lookup
- Then an HTTP GET request

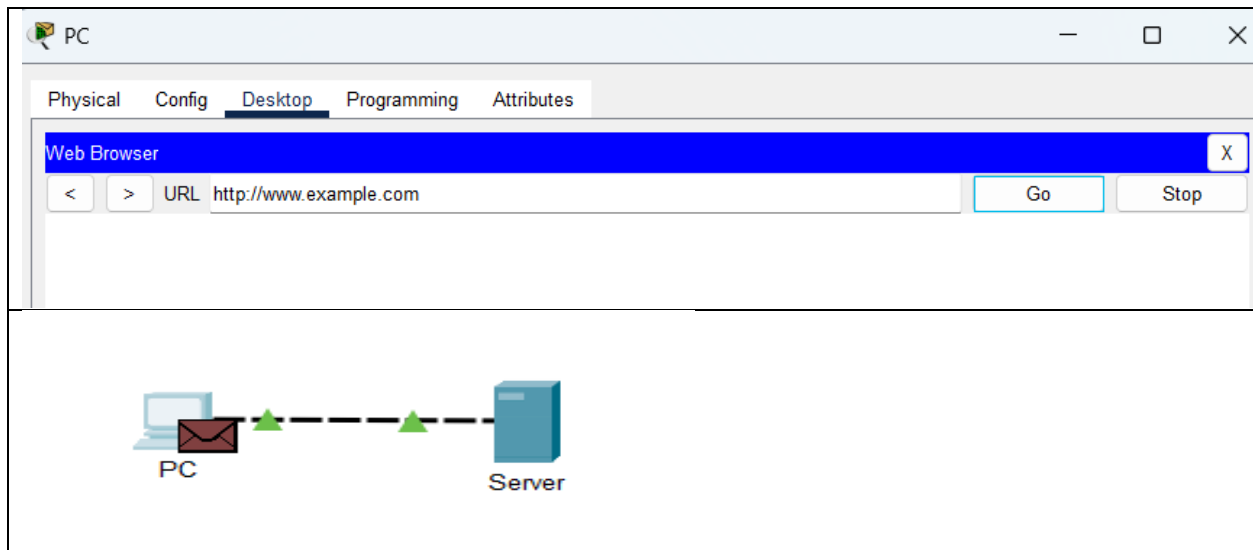
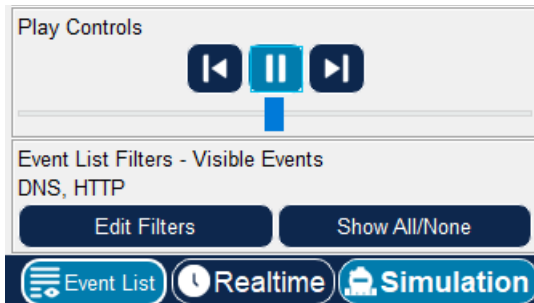


Figure 4: Simulated PC submitting website request to www.example.com

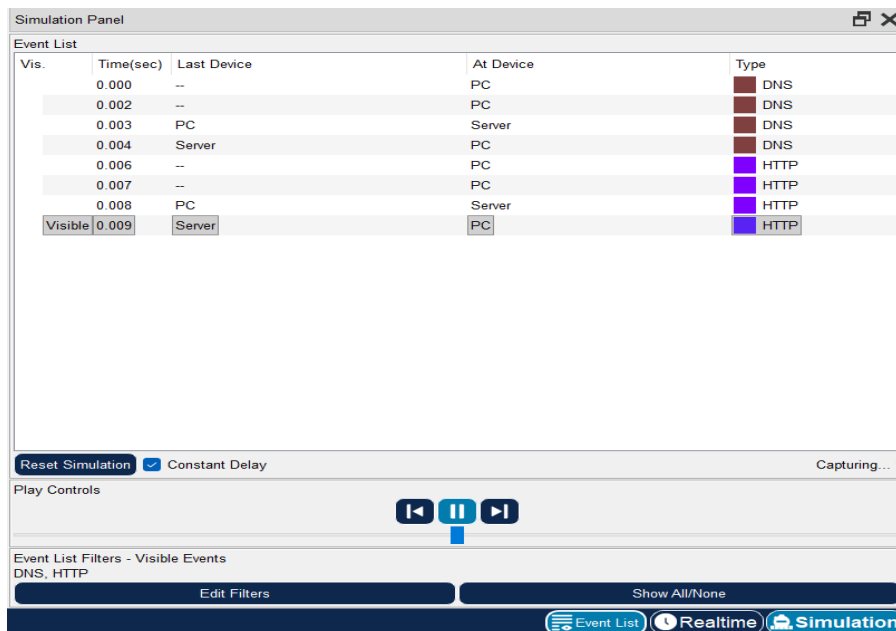
6. Part 4 – Run the Simulation

Steps:

1. In Simulation Panel, click **Play**.



2. Watch the packets flow:
 - DNS query → server
 - DNS response → PC
 - HTTP GET → server
 - HTTP DATA (segments 1 and 2)
 - TCP ACKs
3. If the buffer fills, click **View Previous Events**.





This models an actual, real-life client–server communication.

7. Part 5 – Inspecting the PDU

Now we inspect each exchange at the OSI layer level.

Steps:

1. In Event List, click the **colored square** next to the first event (DNS request).

Simulation Panel				
Event List				
Vis.	Time(sec)	Last Device	At Device	Type
	0.000	--	PC	 DNS

2. The **PDU Information window** opens.
3. Click **Next Layer >>** repeatedly to observe how each OSI layer processes the packet.

PDU Information at Device: PC

OSI Model

Outbound PDU Details

At Device: PC
Source: PC
Destination: 192.168.1.2

In Layers
Layer7
Layer6
Layer5
Layer4
Layer3
Layer2
Layer1

Out Layers
Layer 7: DNS
Layer6
Layer5
Layer 4: UDP Src Port: 1025, Dst Port: 53
Layer 3: IP Header Src. IP: 192.168.1.1, Dest. IP: 192.168.1.2
Layer 2:
Layer1

1. The DNS client sends an A DNS query to the DNS server.

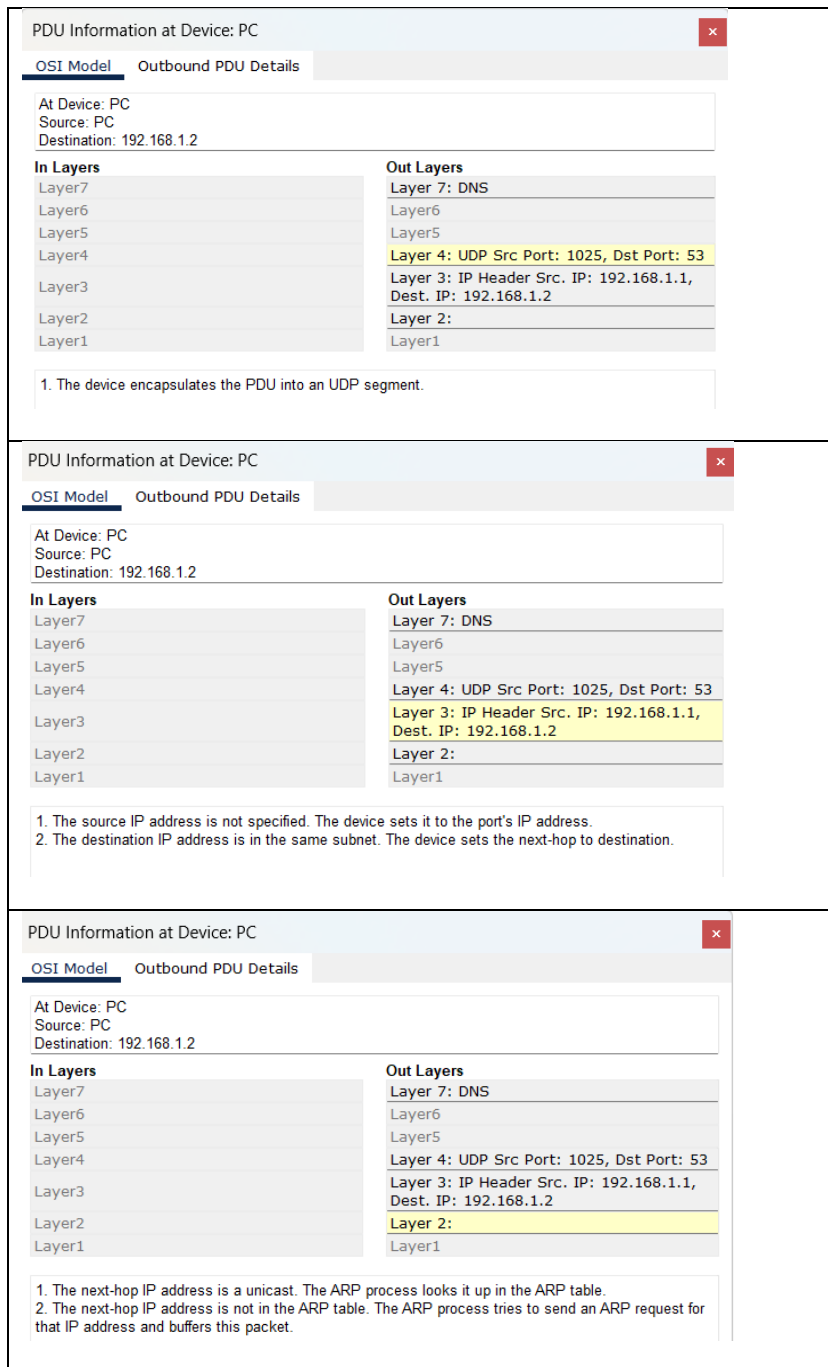


Figure 5: Detailed OSI-layer view of the DNS request packet.

8. Part 6 – Understanding the Client Interaction

DNS Step

When the PC enters www.example.com, it does not know the IP address.

So it sends a:

DNS QUERY: “What is the IP of www.example.com?”

The server replies:

DNS RESPONSE: “The IP is 10.0.0.10”

Only after this can the PC contact the web server.

HTTP Step

Once the PC knows the IP, it sends:

HTTP GET /index.html

The server responds by splitting the webpage into segments:

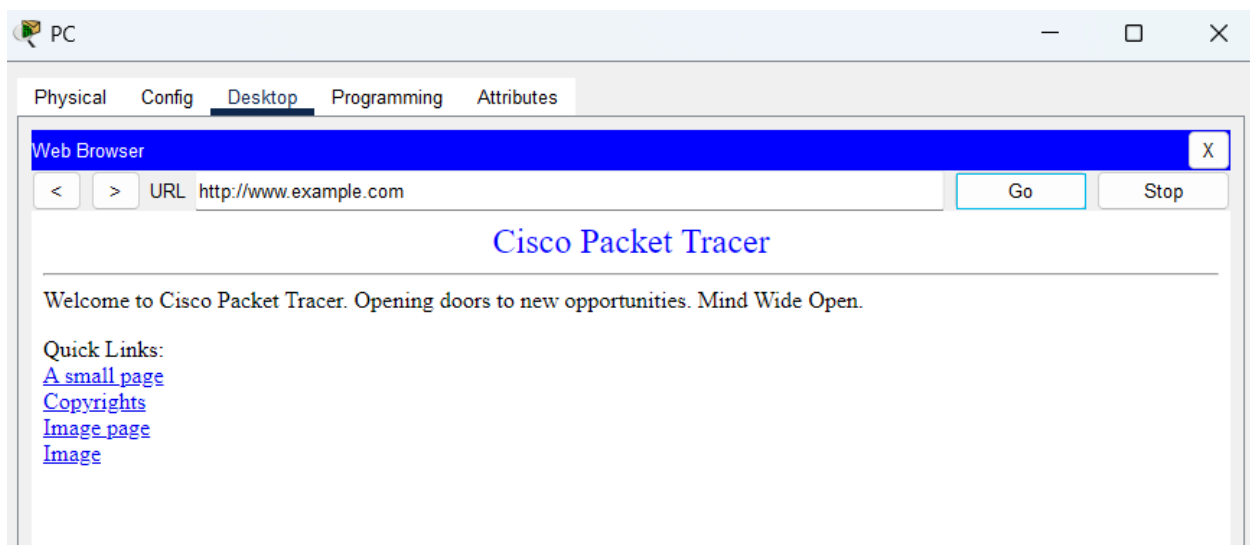
HTTP DATA segment 1

HTTP DATA segment 2

The PC acknowledges each:

TCP ACK

Then the webpage is displayed on the browser.



Understanding DNS and HTTP

DNS – How the Computer Finds the Server

When I enter a website name like www.example.com, my computer doesn't instantly know where that website is located. Computers don't work with names; they work with **IP addresses**. So before anything else happens, the computer must ask a DNS server to translate the name into the correct IP address.

This process works like looking up a contact: you know someone's name, but you need their number.

Similarly, the computer sends a **DNS query** asking:

"What is the IP address of www.example.com?"

The DNS server responds with something like:

"The IP address is 10.0.0.10."

Only once the computer has this IP address can it start communicating with the actual web server.

In simple terms: DNS converts a website name into the numeric address the computer needs in order to reach the server.

HTTP – How the Webpage Is Requested and Delivered

After the DNS step gives the computer the correct IP address, the next part involves actually retrieving the webpage. This is done using a protocol called **HTTP**.

The computer contacts the web server and sends an HTTP request:

"GET /index.html"

which simply means:

"Please send me your main webpage."

The server responds by sending the webpage back, but not in one big piece.

Instead, it breaks the webpage into multiple smaller **TCP segments**, sending them one after another.

For each segment, the computer sends back a **TCP acknowledgment** to confirm it arrived safely.

This process continues until the computer has received every part of the webpage. Once all pieces are collected, the computer reassembles them and displays the full website in the browser.

In simple terms: HTTP is how the computer asks the server for a webpage and how the server sends it back in small, reliable pieces.

- **DNS:** Finds the server's IP address.
- **HTTP:** Requests the webpage and receives it in segments.

9. Summary & Reflection

What I learned:

- DNS converts domain names to IP addresses (critical for internet use).
- HTTP retrieves webpage data using segmented TCP delivery.
- OSI layers control how data is packaged, transmitted, routed, received, and displayed.
- Packet Tracer provides an excellent view of real packet flow in a simplified environment.

Why this matters:

Every time you visit a website in real life, your device performs the exact steps seen in this simulation DNS, HTTP GET, segmentation, acknowledgements, and webpage rendering.